# CP2406

# Assessment Task 3, Option 3 (30%)

## Improve a Barnes-Hut Galaxy Simulation in C++

*Help Garry make the simulation safer, faster and easier to investigate*

| Year | Assessment | Weighting |
|---|---|---|
| 2026 | Individual | 30% |
| Platform | Deliverables | Video |
| CLion / C++ | Project ZIP + CODE_REVIEW.md + video | 3-5 minutes |

**NOTE: Complete this assessment individually. The simulation is an educational software model. Your work is assessed for C++ design, validation, testing, reporting and organisation, not astrophysics expertise.**

## The Scenario

Congratulations! You are a junior C++ developer working with Garry, an astrophysics researcher. Garry has inherited a Barnes-Hut N-body simulation that organises bodies in an octree, approximates gravitational interactions and renders image frames. The prototype runs, but its collision processing, configuration, reporting and program structure need systematic improvement.

You do not need to redesign the Barnes-Hut mathematics. Your focus is C++ algorithm design, containers and pointers, parameter validation, reproducible experiments, testing and software organisation.

## Understanding the Starter Project

- Bhtree.cpp/.h stores octree nodes, inserts bodies and approximates gravitational interactions.
- Octant.cpp/.h represents a 3D region and creates its eight child octants.
- Types.h defines vec3, body, StarSpec and DiskKinematics.
- Constants.h contains the baseline simulation, physics, integration, rendering and collision settings.
- Collisions.h contains collision-related functions that require redesign and testing.
- main.cpp currently coordinates initialisation, simulation, rendering and output.

# What Are You Being Asked to Do?

Complete all four technical tasks and the weekly development journal. Tasks 1-3 are worth 6 marks each, Task 4 is worth 7 marks, and the weekly journal is worth 5 marks. Keep the original Barnes-Hut force calculation working while improving the surrounding software.

## Task 1: Configurable Simulation and Parameter Investigation (6 marks)

- Create SimulationConfig.h and SimulationConfig.cpp to hold user-adjustable settings rather than adding more preprocessor macros.
- Validate body count, system size, time step, step count, collision threshold and collision sample count. Reject invalid values with clear exceptions or error messages.
- Provide command-line options for at least --bodies, --system-size, --steps, --dt and --collision-threshold, plus --help.
- Run a controlled investigation that changes at least three parameters while keeping the other settings fixed.
- Record each configuration and explain visible or measured effects. Include one deliberately invalid configuration and show how it is handled.

## Task 2: Collision Detection and Safe Merging (6 marks)

- Move collision declarations to Collisions.h and implementations to Collisions.cpp.
- Implement checkAndMergeCollision(body& a, body& b, double thresholdAU). It must ignore retired bodies and self-pairs, compare distance with the threshold, conserve total mass and linear momentum, update the merged position using centre of mass, and retire the second body safely.
- Implement a collision pass that examines a configurable number of candidate pairs using std::mt19937 and std::uniform_int_distribution. A pair must contain two distinct valid indices.
- Prevent non-terminating loops when collisions are rare or disabled. The requested sample count limits attempts rather than successful collisions.
- Return CollisionStats containing attempted pairs, valid pairs, merged pairs and active bodies after the pass. Make seeded runs reproducible.

### Pseudocode for checkAndMergeCollision()

```
Function checkAndMergeCollision(a, b, thresholdAU):
    If a and b refer to the same body: return false
    If a is retired or b is retired: return false
    Validate that thresholdAU is not negative
    distance = Euclidean distance between a.position and b.position
    If distance is greater than thresholdAU: return false
    totalMass = a.mass + b.mass
    If totalMass is not positive: handle as an invalid merge
    mergedPosition = (a.position × a.mass + b.position × b.mass) / totalMass
    mergedVelocity = (a.velocity × a.mass + b.velocity × b.mass) / totalMass
    Update a with totalMass, mergedPosition and mergedVelocity
    Retire b safely so it is ignored by later processing
    Return true
```

### Pseudocode for the bounded collision pass

```
Function runCollisionPass(bodies, sampleCount, thresholdAU, seed):
    Validate sampleCount and thresholdAU
    Create mt19937 random generator using seed
    Create a uniform index distribution for the body collection
    Initialise CollisionStats to zero
    Repeat at most sampleCount times:
        Increase attemptedPairs
        Select indexA and indexB
        If the indices are equal: continue
        If either index is invalid or either body is retired: continue
        Increase validPairs
        If checkAndMergeCollision(bodyA, bodyB, thresholdAU) is true:
            Increase mergedPairs
    Count the bodies that are still active
    Store the active-body count in CollisionStats
    Return CollisionStats
```

*The requested sample count limits attempts, not successful collisions. This ensures the pass terminates when collisions are rare or disabled.*

## Task 3: Reporting, Refactoring and Reproducible Experiments (6 marks)

- Create Simulation.cpp/.h to own initialisation and simulation-step responsibilities, Renderer.cpp/.h for PPM output, and CsvReporter.cpp/.h for experiment summaries. Keep main.cpp as the coordinator.
- Write a CSV row for every experiment containing the selected configuration, random seed, elapsed time, collision statistics, active-body count, total mass, barycentre position and status.
- Run at least three collision thresholds and two body-count settings on the same baseline configuration. Repeat at least one seeded configuration to demonstrate reproducibility.
- Compare collision-disabled, rare-collision and higher-collision settings. Explain runtime, active-body count, mass conservation, centre-of-mass movement and visible output.
- Ensure ranking or ordering of experiment rows is deterministic, for example by experiment name and seed.

## Task 4: Code Review, Refactoring and Testing (7 marks)

- Submit a CODE_REVIEW.md file using the structure provided in Appendix A. Replace every prompt with your own explanation and refer to specific files, functions and code from your submitted project. Do not include generic or copied responses. Answer all questions below in your own words.
- Use meaningful names, const where appropriate, safe indexing, standard random-number facilities, appropriate exceptions, RAII and comments that explain decisions rather than obvious syntax.
- Add at least four focused tests or test cases covering collision and configuration edge cases. Manual test evidence is acceptable if inputs, expected results and actual results are clearly recorded.
- Include a brief AI-use statement describing the tool, purpose and how generated suggestions were checked.

## Code Review Questions

1. For body* a, what does a->position.x mean, and how is it related to (*a).position.x?

2. How does Bhtree::insertIntoChild decide which of the eight child octants receives a body? What boundary behaviour should a reviewer notice?

3. Why should a collision attempt select two distinct indices, and what can go wrong if the same body is selected twice?

4. Why must a sampling loop be bounded by attempts rather than by the number of successful collisions?

5. How do the centre-of-mass position and momentum equations preserve physical quantities during a merge?

6. Why is std::mt19937 with an explicit seed preferable to rand() for the experiment requirement?

7. Which responsibilities were moved out of main.cpp, and how does the new structure make the program easier to test?

8. How did you test zero or negative parameters, zero collision threshold, exact-threshold distance, retired bodies and deterministic seeded runs?

## What You Do Not Need to Do

- Change the Barnes-Hut force approximation or derive its physics.
- Create a graphical user interface.
- Produce scientifically accurate galaxy research results.
- Use every optional parameter in Constants.h.
- Replace the PPM renderer unless your refactoring requires a small interface change.

## What Should You Deliver?

### 1. Improved CLion Project

- A clean ZIP containing source files, headers, CMakeLists.txt, tests or test evidence, experiment CSV, CODE_REVIEW.md and the completed weekly project development journal.
- Remove cmake-build-* folders, IDE caches, generated frame sequences and temporary files.
- Recommended name: CP2406-A3-Option3-FirstName-LastName-Code.zip

### 2. Weekly Project Development Journal

- Complete the three weekly entries and final reflection in Appendix B during development. Each entry must be dated and include specific code changes, evidence, testing, problem-solving, assistance used and the next planned step. The completed journal is worth 5 marks.

### 3. Video Demonstration

- A 3-5 minute demonstration of your own solution.
- Show --help, one valid run, one invalid configuration, collision statistics, the experiment CSV and one before/after refactoring example.
- Explain one collision edge case and one seeded reproducibility test.

- Recommended name: CP2406-A3-Option3-FirstName-LastName-Video.mp4

## Submission Checklist

☐ Project builds from a clean CLion configuration.

☐ Original Barnes-Hut simulation still runs.

☐ CLI validation and help output work.

☐ Collision attempts are bounded and do not hang when collisions are absent.

☐ CSV includes every required field and experiment.

☐ At least four meaningful tests are documented.

☐ CODE_REVIEW.md answers all questions and contains an AI-use statement.

☐ Video shows the required evidence.

# Rubric for CP2406 Assessment Task 3, Option 3

Total: 30 marks (Technical project: 25 marks; Weekly journal: 5 marks). The structure and performance bands match the other 2026 Assessment Task 3 options.

| Criterion | Excellent | Good | Satisfactory | Needs Improvement | Not Evident |
|---|---|---|---|---|---|
| **T1. Configuration/CLI (2.5)** | Complete class, required options and clear help. | Mostly complete; minor issue. | Basic options work. | Incomplete or tightly coupled. | Not implemented. |
| **T1. Validation (1.5)** | Robust checks and clear failure messages. | Most invalid cases handled. | Basic checks. | Weak or inconsistent. | Not evident. |
| **T1. Investigation (2)** | Three or more controlled changes with insightful evidence. | Required changes with good comparison. | Some parameter evidence. | Limited or unsupported. | Not submitted. |
| **T2. Merge correctness (2.5)** | Correct distance, mass, centre of mass, momentum and retirement. | Mostly correct; minor edge case. | Basic merge works. | Partial or unsafe. | Not implemented. |
| **T2. Bounded sampling (2.5)** | Distinct valid pairs, seeded RNG, bounded attempts and accurate stats. | Good with minor omission. | Basic sampling works. | Fragile, biased or may hang. | Not implemented. |
| **T2. Design/quality (1)** | Cohesive interface, const/safe practices and decision comments. | Good with minor issue. | Usable structure. | Poor separation. | Not evident. |
| **T3. Refactoring (2)** | Clear Simulation, Renderer and Reporter responsibilities; thin main. | Good separation with minor issue. | Some useful separation. | Limited or inconsistent. | Not evident. |
| **T3. CSV/metrics (2)** | Complete, correct and deterministic experiment output. | Mostly complete. | Basic report. | Incomplete or confusing. | Not produced. |

| Criterion | Excellent | Good | Satisfactory | Needs Improvement | Not Evident |
|---|---|---|---|---|---|
| **T3. Experiments/analysis (2)** | All required comparisons plus repeatability and insightful analysis. | Required tests with good comparison. | Some experiments. | Limited evidence. | Not submitted. |
| **T4. Review responses (3)** | All eight answers accurate, specific and clearly linked to code. | Mostly accurate. | Basic understanding. | Several errors or gaps. | Not submitted. |
| **T4. Tests (2.5)** | Four or more meaningful tests including edge and reproducibility cases. | Good tests with minor omission. | Basic evidence. | Minimal or unclear. | No tests. |
| **T4. Quality/AI statement (1.5)** | Consistent safe C++, useful comments and transparent verified AI statement. | Good with minor issue. | Basic quality and statement. | Inconsistent or incomplete. | Not evident. |

## Weekly Journal Rubric (5 marks)

| Criterion | Excellent | Good | Satisfactory | Needs Improvement | Not Evident |
|---|---|---|---|---|---|
| J1. Three weekly entries (2) | All three entries are dated, specific and show clear progressive development with relevant evidence. | Three useful entries with only minor gaps. | Entries show basic progress but lack detail or evidence in places. | Entries are incomplete, vague or mainly retrospective. | Not submitted. |
| J2. Testing and problem-solving (1.5) | Specific tests, expected and actual results, and well-explained problem-solving are documented across the | Good testing and problem-solving evidence with minor omissions. | Basic tests and one problem-solving process are recorded. | Minimal or unclear testing and problem-solving evidence. | Not evident. |

| | | | | | |
|---|---|---|---|---|---|
| | project. | | | | |
| J3. Reflection, evidence and assistance log (1.5) | Evidence is authentic and well explained; reflections show strong understanding; AI and external help are transparently recorded and verified. | Good evidence, reflection and assistance records with minor omissions. | Basic evidence and reflection; assistance is recorded. | Weak evidence, reflection or incomplete assistance disclosure. | Not evident. |

## Final Advice

Test checkAndMergeCollision with two manually created bodies before running the full simulation. Then test the bounded sampling pass with a tiny body collection and a fixed seed. This separates algorithm debugging from rendering and Barnes-Hut behaviour.

# Appendix A: CODE_REVIEW.md Template

Copy this structure into a file named CODE_REVIEW.md in the top-level CLion project folder. Replace every prompt with your own response and refer to specific files, classes, functions and code from your submitted project. Generic or copied responses do not demonstrate code understanding.

## 1. Student and Project Details

**Student name:** [Enter your name]

**Student ID:** [Enter your student ID]

**Project option:** Galaxy Simulation

## 2. Pointer Member Access

**Question:** For body* a, what does a->position.x mean, and how is it related to (*a).position.x?

**Your response:** [Explain pointer dereferencing and member access.]

**Code evidence:** File: ___ Function: ___ Fragment: ___

## 3. Octree Child Selection

**Question:** How does Bhtree::insertIntoChild decide which child octant receives a body? What boundary behaviour should a reviewer notice?

**Your response:** [Refer to the supplied implementation.]

**Code evidence:** ___

## 4. Distinct Collision Indices

**Question:** Why must a collision attempt select two distinct indices, and what can go wrong if the same body is selected twice?

**Your response:** [Explain the safety and validity issue.]

**Code evidence:** ___

## 5. Bounded Sampling

**Question:** Why must the sampling loop be bounded by attempts rather than successful collisions?

**Your response:** [Explain termination when collisions are rare or disabled.]

**Code evidence:** ___

## 6. Conservation During Merge

**Question:** How do the centre-of-mass position and momentum equations preserve physical quantities during a merge?

**Your response:** [Use the variables and calculations in your implementation.]

**Code evidence:** ___

## 7. Reproducible Randomness

**Question:** Why is std::mt19937 with an explicit seed preferable to rand() for these experiments?

**Your response:** [Explain reproducibility and experiment comparison.]

**Evidence from repeated run:** ___

## 8. Refactoring Summary

**Question:** Which responsibilities were moved out of main.cpp, and how does the new structure make the program easier to test?

**Simulation responsibilities:** ___

**Renderer responsibilities:** ___

**CsvReporter responsibilities:** ___

**Other change:** ___

## 9. Testing Evidence

Document at least four focused tests, including zero or negative parameters, zero collision threshold, exact-threshold distance, retired bodies and deterministic seeded runs.

| Test | Purpose | Input/setup | Expected result | Actual result | Pass/fail and code location |
|---|---|---|---|---|---|
| Zero or negative parameter | [Complete] | [Complete] | [Complete] | [Complete] | [Complete] |
| Zero collision threshold | [Complete] | [Complete] | [Complete] | [Complete] | [Complete] |
| Exact-threshold distance | [Complete] | [Complete] | [Complete] | [Complete] | [Complete] |
| Retired body | [Complete] | [Complete] | [Complete] | [Complete] | [Complete] |
| Deterministic seeded run | [Complete] | [Complete] | [Complete] | [Complete] | [Complete] |

## 10. Challenge and Resolution

**Challenge encountered:** [Your response]

**How I investigated it:** [Your response]

**Solution implemented:** [Your response]

**Evidence that the solution works:** [Your response]

## 11. Generative AI Use Statement

Complete Option A or Option B.

**Option A: Generative AI was used**

**Tool or tools used:** [Your response]

**Purpose of use:** [Your response]

**Parts of the project affected:** [Your response]

**How I checked the output:** [Your response]

**Changes I made:** [Your response]

**How I demonstrated my understanding:** [Your response]

**Option B: Generative AI was not used**

I did not use generative AI to produce or revise the submitted code, documentation or analysis for this assessment.

## 12. Final Student Declaration

☐ The explanations are written in my own words.

☐ The code references relate to my submitted project.

☐ The reported tests were performed on my implementation.

☐ I can explain and modify the submitted code.

☐ Any generative AI use has been declared accurately.

**Student name:** _____ **Date:** _____

## Appendix B: Weekly Project Development Journal

Complete the following journal during development. The journal is worth 5 marks and forms part of the assessment submission.

### CP2406

# Weekly Project Development Journal

*Reusable for Emotion Smoother, Football Sustained Risk, Galaxy Simulation and other approved C++ projects*

**Student name:** _____

**Student number:** _____

**Project option:** _____

**Teaching period:** _____

Purpose

Use this journal to show progressive, authentic development of your project. Complete one entry at each required checkpoint. Entries should identify the code you changed, evidence collected, problems solved, tests performed, assistance used and your next step.

## Submission Instructions

- Complete the journal during development, not only at the end of the assignment.
- Use specific file, class and function names. For example: main.cpp, EmotionSmoother, risk calculation, simulation update loop or CMake configuration.
- Include dated evidence from your own project. Suitable evidence may include CLion screenshots, terminal output, test results or saved project versions.
- Record generative AI use and other assistance honestly. Explain how suggestions were checked, changed or rejected.
- Submit the completed journal in the format and at the checkpoints specified by your subject coordinator.

## Suggested Checkpoints

| Entry | Suggested focus | Evidence examples |
|---|---|---|

| | | |
|---|---|---|
| 1 | Understand starter project; plan and begin the first core feature. | Clean build, codebase notes, initial input/output or function. |
| 2 | Implement and test the main algorithm or simulation behaviour. | Changed functions, targeted tests, debugging evidence. |
| 3 | Integrate features; compare settings or scenarios; refactor and test. | Comparison output, edge cases, code-quality changes. |
| Final reflection | Summarise learning, verification, AI use and readiness to explain the code. | Completed journal, final test summary and submission check. |

# Weekly Journal Entry 1

**Week / dates:** _____

**Project option:** ☐ Emotion Smoother ☐ Football Sustained Risk ☐ Galaxy Simulation ☐ Other: _____

**Checkpoint / task focus:** _____

## 1. Goals for This Week

**What did you plan to complete? State specific functions, features, tests or documentation.**

## 2. Development Work Completed

| File / component | Function or area changed | Summary of your own changes |
|---|---|---|

| File / component | Function or area changed | Summary of your own changes |
|---|---|---|

## 3. Evidence of Progress

Attach or insert at least one dated item that shows genuine progress, such as a CLion screenshot, program output, test result or version-history copy. Do not include private information.

**Evidence filename / link:** _____

**Explain what the evidence shows and how it relates to this week's work.**

## 4. Problem-Solving Record

**Describe one problem, error or design decision you encountered.**

**Explain how you investigated it, what you changed, and why the final approach was selected.**

## 5. Testing Performed

| Test / input | Expected result | Actual result | Pass / fail and action |
|---|---|---|---|

## 6. Generative AI and External Help Log

Record all assistance used this week. If none was used, tick the statement below. Do not paste full AI conversations; summarise the purpose and your verification.

| Tool / person | Purpose or question | Suggestion used or rejected | How you checked / changed it |
|---|---|---|---|
| | | | |

☐ No generative AI or external help was used for this week's work.

## 7. Weekly Reflection and Next Step

**What did you learn, and which part of the work can you confidently explain or modify?**

**What is the next specific task you will complete?**

**Student declaration:** This entry accurately records my progress and assistance received.

**Student initials / signature:** _____ Date: _____

# Weekly Journal Entry 2

**Week / dates:** _____

**Project option:** ☐ Emotion Smoother ☐ Football Sustained Risk ☐ Galaxy Simulation ☐ Other: _____

**Checkpoint / task focus:** _____

## 1. Goals for This Week

**What did you plan to complete? State specific functions, features, tests or documentation.**

## 2. Development Work Completed

| File / component | Function or area changed | Summary of your own changes |
|---|---|---|

## 3. Evidence of Progress

Attach or insert at least one dated item that shows genuine progress, such as a CLion screenshot, program output, test result or version-history copy. Do not include private information.

**Evidence filename / link:** _____

**Explain what the evidence shows and how it relates to this week's work.**

## 4. Problem-Solving Record

**Describe one problem, error or design decision you encountered.**

**Explain how you investigated it, what you changed, and why the final approach was selected.**

## 5. Testing Performed

| Test / input | Expected result | Actual result | Pass / fail and action |
|---|---|---|---|

## 6. Generative AI and External Help Log

Record all assistance used this week. If none was used, tick the statement below. Do not paste full AI conversations; summarise the purpose and your verification.

| Tool / person | Purpose or question | Suggestion used or rejected | How you checked / changed it |
|---|---|---|---|

☐ No generative AI or external help was used for this week's work.

## 7. Weekly Reflection and Next Step

**What did you learn, and which part of the work can you confidently explain or modify?**

**What is the next specific task you will complete?**

**Student declaration:** This entry accurately records my progress and assistance received.

**Student initials / signature:** _____ Date: _____

# Weekly Journal Entry 3

**Week / dates:** _____

**Project option:** ☐ Emotion Smoother ☐ Football Sustained Risk ☐ Galaxy Simulation ☐ Other: _____

**Checkpoint / task focus:** _____

## 1. Goals for This Week

**What did you plan to complete? State specific functions, features, tests or documentation.**

## 2. Development Work Completed

| File / component | Function or area changed | Summary of your own changes |
|---|---|---|

## 3. Evidence of Progress

Attach or insert at least one dated item that shows genuine progress, such as a CLion screenshot, program output, test result or version-history copy. Do not include private information.

**Evidence filename / link:** _____

**Explain what the evidence shows and how it relates to this week's work.**

## 4. Problem-Solving Record

**Describe one problem, error or design decision you encountered.**

**Explain how you investigated it, what you changed, and why the final approach was selected.**

## 5. Testing Performed

| Test / input | Expected result | Actual result | Pass / fail and action |
|---|---|---|---|

## 6. Generative AI and External Help Log

Record all assistance used this week. If none was used, tick the statement below. Do not paste full AI conversations; summarise the purpose and your verification.

| Tool / person | Purpose or question | Suggestion used or rejected | How you checked / changed it |
|---|---|---|---|

☐ No generative AI or external help was used for this week's work.

## 7. Weekly Reflection and Next Step

**What did you learn, and which part of the work can you confidently explain or modify?**

**What is the next specific task you will complete?**

**Student declaration:** This entry accurately records my progress and assistance received.

**Student initials / signature:** _____ Date: _____

# Final Project Reflection

**Project option:** ☐ Emotion Smoother ☐ Football Sustained Risk ☐ Galaxy Simulation ☐ Other: _____

## 1. Development Summary

**Summarise how the project developed across the journal checkpoints. Identify the most important features you personally implemented.**

## 2. Most Difficult Problem

**Describe the most difficult technical or design problem and how you resolved it.**

## 3. Most Valuable Test

**Identify the test that gave you the strongest confidence in the project, including expected and actual results.**

## 4. Code Understanding

**Name one function you could explain and modify during a live demonstration. Describe its purpose and key decisions.**

## 5. Final AI-Use Summary

**List all generative AI tools used across the assignment, their purpose, and how outputs were verified, modified or rejected. If none were used, state this clearly.**

## 6. Future Improvement

**If you continued the project, what would you improve next and why?**

## Final Submission Check

☐ Journal entries are complete and dated.

☐ Evidence is included and relates to the stated work.

☐ AI and external help are transparently recorded.

☐ Project builds from a clean configuration.

☐ I can explain and modify the submitted code.

☐ Project ZIP excludes build folders, caches and unnecessary generated files.

**Student signature:** _____ Date: _____